

AI Engineer Technical Assignment

Multi-Agent Reasoning System for PDF Analysis

Objective

You are required to build a modular, agent-driven application that enables semantic interaction with arbitrary PDF documents through a web interface. The system should support both contextual question answering and abstractive summarization. The design must integrate Retrieval-Augmented Generation (RAG), agent-based orchestration, and an interactive frontend powered by Streamlit. The implementation must leverage AgentSDK and a capable large language model (LLM) backend.

Problem Description

Develop a Streamlit application that allows end-users to upload **one or more PDF documents** and interact with them conversationally. The user may ask questions about the content (including queries spanning multiple documents) or request a summary of one or all uploaded documents.

The application must interpret user intent autonomously and route the request to the appropriate agent without requiring manual input mode selection.

Three types of agents must be instantiated using AgentSDK:

1. **Retrieval-Augmented Generation (RAG) Agent**
Responsible for handling user questions by retrieving relevant sections from the document(s) and generating informed responses grounded in the retrieved context.
2. **Summarization Agent**
Capable of generating a coherent, high-level summary of the uploaded document's content (single or multi-document), abstracting from individual sections while maintaining factual integrity.
3. **Specialized Reasoning Agent(s)**
Designed to perform higher-order reasoning tasks using the outputs of the RAG Agent.
Examples include:
 - **Comparator Agent** – compares and contrasts findings across documents or sections.
 - **Timeline Builder Agent** – arranges events chronologically across documents.
 - **Aggregator Agent** – merges overlapping evidence and removes redundancy.

The system's **planner** must dynamically decide which agent or chain of agents to invoke based on user intent.

For instance:

- *Planner → RAG Agent → Comparator Agent → Final Output*
- *Planner → RAG Agent → Timeline Agent → Final Output*
- *Planner → Summarization Agent → Final Output*

The user should not manually choose between these agents; the system should decide dynamically based on input semantics.

In addition, the system must:

- **Support agent-to-agent reasoning (chain of agents)**, allowing the planner to call the RAG agent first to retrieve evidence and then call one or more specialized reasoning agents before producing the final answer.
- **Cite supporting evidence** in every answer, including a ranked list of source passages with document name, page number, chunk ID, and similarity score.
- **Provide a document navigator** so users can visually explore the PDF(s) and see where the cited passages come from.

Technical Requirements

Frontend

- Use **Streamlit** to develop the interface.
- Provide a **file uploader** for **multiple PDF documents**.
- Allow **freeform natural language input** from the user.
- Do **not** include explicit controls (e.g., toggles or buttons) to switch between query and summarization modes.
- The system must **infer user intent** and delegate to the appropriate agent(s).
- Maintain an **interactive chat interface** that logs user prompts and agent responses.
- Include visible **progress indicators** during agent execution.

- Implement a **document navigator**:
 - Display the uploaded PDF(s) in an interactive viewer.
 - Highlight the cited passages corresponding to evidence returned by the backend.
 - Allow users to click citations in the chat to scroll directly to the referenced section in the viewer, showing document name, page, chunk ID, and similarity score.

Backend and Agent Logic

PDF Ingestion and Processing:

- Extract text from PDFs using tools such as `pdfplumber`, `PyMuPDF`, or equivalent.
- Segment the document into meaningful chunks using an appropriate text-splitting strategy.
- Convert chunks into embeddings using a transformer-based model.
- Store and index embeddings using a vector database (FAISS, Chroma, or equivalent).
- The indexing layer must support **multi-document storage** and **cross-document retrieval**.

RAG Agent:

- Embed user queries and retrieve top-k relevant document segments across all uploaded PDFs.
- Pass the retrieved context and the query to an LLM to generate a grounded response.
- Include context management, fallback handling, and clear output formatting.
- Return a **ranked list of supporting evidence** for each response, including:
 - Document name
 - Page number
 - Chunk ID

- Retrieval/similarity score
- A short excerpt of the passage
- The final answer must reference these evidence items (e.g., with numbered citations linking to the navigator).

Summarization Agent:

- Implement a summarization strategy appropriate for long or multiple documents (map-reduce, chunk-refinement, etc.).
- Ensure coherence and factual consistency across sections.
- Avoid redundancy and preserve key contextual details.

Specialized Reasoning Agent(s):

- Must take the **retrieved evidence** from the RAG Agent as input.
- Perform higher-level reasoning tasks such as comparison, timeline construction, or evidence aggregation.
- Return a structured output that the planner can integrate into the final LLM response.

Planner / AgentSDK Orchestration:

- The planner must autonomously:
 - Detect user intent (e.g., query, summarization, comparison).
 - Determine which agent(s) to invoke and in what sequence.
 - Produce a reasoning trace showing the agent chain used.
- Agents should be **modular and extendable**, allowing easy addition of future reasoning tools.

Deadline

All submissions must be completed and submitted no later than **2 days** from the time of assignment. Extensions will not be granted except under exceptional circumstances.

